AI ASSITED CODING

LAB ASSIGNMENT-10.1

Name: Deepthi

Ht.No: 2303A52390

Batch: 32

Code Review and Quality: Using AI to Improve Code Quality and Readability

Lab Objectives

• Use AI for automated code review and quality enhancement.

• Identify and fix syntax, logical, performance, and security issues in Python code.

• Improve readability and maintainability through structured refactoring and comments.

• Apply prompt engineering for targeted improvements.

• Evaluate AI-generated suggestions against PEP 8 standards and software engineering best practices

Lab Outcomes

1. Students will be able to use AI tools to review code.

2. Students will be able to improve code quality and readability.

3. Students will be able to identify and fix common coding issues.

Task Description #1 – Syntax and Logic Errors

Task: Use AI to identify and fix syntax and logic errors in a faulty

Python script.

Sample Input Code:

Calculate average score of a student

def calc_average(marks):

total = 0

for m in marks:

total += m

average = total / len(marks)

return avrage # Typo here

marks = [85, 90, 78, 92]

print("Average Score is ", calc_average(marks)

Expected Output:

• Corrected and runnable Python code with explanations of the

fixes.

Code:

```
#refactore the below code to fix the typo and make it more efficient
def calc_average(marks):
    total = sum(marks)
    average = total / len(marks)
    return average
marks = [85, 90, 78, 92]
print("Average Score is ", calc_average(marks))
```

Output:

```
PS C:\Users\HP\OneDrive\Documents\AIAC_2026> & C:/Users/HP/AppData/Local/Programs/Python/Python313/python.exe c:/Users/HP/OneDrive/Documents/AIAC_2026/Lab
Average Score is  86.25
```

Explanation:

In this python program we fixed the typo: changed return avrage → return average.

Used Python’s built-in sum() instead of a manual loop, making the code shorter and more efficient.

Added the missing closing parenthesis in the print() statement.

Task Description #2 – PEP 8 Compliance

Task: Use AI to refactor Python code to follow PEP 8 style guidelines.

Sample Input Code:

def area_of_rect(L,B) : return L*B

print(area_of_rect(10,20))

Expected Output:

• Well-formatted PEP 8-compliant Python code.

Code:

```
#refactored the below code to fix the typo and make it more efficient
def area_of_rect(length,breadth) :
    return length*breadth
print(area_of_rect(10,20))
```

Output:

```
PS C:\Users\HP\OneDrive\Documents\AIAC_2026> &
200
```

Explanation:

In this python code the Function name changed to area_of_rect (descriptive, lowercase with underscores).Parameters renamed to length and breadth for clarity.Added proper spacing and line breaks for readability.Removed unnecessary spaces before colons and parentheses.

Task Description #3 – Readability Enhancement

Task: Use AI to make code more readable without changing its logic.

Sample Input Code:

def c(x,y):

return x*y/100

a=200

b=15

print(c(a,b))

Expected Output:

• Python code with descriptive variable names, inline

comments, and clear formatting.

Code:

```
#refactor the below code to fix the typo and make it more efficient
def calculate_percentage(x,y):
    return x*y/100 # calculate the percentage of x with respect to y
a=200
b=15
print(calculate_percentage(a,b)) # output: 30.0
```

Output:

```
PS C:\Users\HP\OneDrive\Documents\AIAC_2026> & C:/Users/HP/AppData/Local/Programs/Py
30.0
```

Explanation:

In this python program we renamed variables (a, b) to descriptive names (x,y) for clarity. Added inline comments to explain each step. Improved formatting with spacing and line breaks for better readability.

Task Description #4 – Refactoring for Maintainability

Task: Use AI to break repetitive or long code into reusable functions.

Sample Input Code:

students = ["Alice", "Bob", "Charlie"]

print("Welcome", students[0])

print("Welcome", students[1])

print("Welcome", students[2])

Expected Output:

• Modular code with reusable functions.

Code:

```
#refactor the below code to fix the typo and make it more efficient
def welcome_student(names):
    for name in names:
        print("Welcome", name)
print(welcome_student(["Alice", "Bob", "Charlie"]))
```

Output:

```
PS C:\Users\HP\OneDrive\Documents\AIAC_2026> & C:/Users/HP/AppData/
Welcome Alice
Welcome Bob
Welcome Charlie
```

Explanation:

In this python program extracted repetitive print statements into a reusable function welcome_student . Used a loop to handle any number of students, making the code scalable and flexible. Removed the unnecessary print() around the function call, since the function itself already prints the messages.

Task Description #5 – Performance Optimization

Task: Use AI to make the code run faster.

Sample Input Code:

Find squares of numbers

nums = [i for i in range(1,1000000)]

squares = []

for n in nums:

squares.append(n**2)

print(len(squares))

Expected Output:

• Optimized code using list comprehensions or vectorized operations.

Code:

```
#optimize the below code using list comprehension or vectorized operations
nums = [i for i in range(1,1000000)]
squares = [n**2 for n in nums]
print(len(squares))
```

Output:

```
PS C:\Users\HP\OneDrive\Documents\AIAC_2026> &
999999
```

Explanation:

In this python program we replaced i for i in range(…) with range(..) to avoid creating an unnecessary intermediate list. Used list comprehension to compute squares in a single, concise step. This reduces memory overhead and improves execution speed compared to the original loop-based approach.

Task Description #6 – Complexity Reduction

Task: Use AI to simplify overly complex logic.

Sample Input Code:

```
def grade(score):
    if score >= 90:
        return "A"
    else:
        if score >= 80:
            return "B"
        else:
            if score >= 70:
                return "C"
            else:
                if score >= 60:
                    return "D"
                else:
                    return "F"
```

Expected Output:

• Cleaner logic using elif or dictionary mapping.

Code:

```
#refactor the below code for cleaner logic using elif or dictionary mapping.
def grade(score):
    if score >= 90:
        return "A"
    elif score >= 80:
        return "B"
    elif score >= 70:
        return "C"
    elif score >= 60:
        return "D"
    else:
        if score >= 80:
            return "B"
        else:
            if score >= 70:
                return "C"
            else:
                if score >= 60:
                    return "D"
                else:
                    return "F"
print(grade(85)) # output: "B"
```

Output:

```
PS C:\Users\HP\OneDrive\Documents\AIAC_2026> &
B
```

Explanation:

In this python program we used elif as it removes redundant nested else blocks, making the logic straightforward.The function now checks conditions in sequence and returns the grade immediately once matched.This improves readability and avoids unnecessary repetition.